

LLD – Search Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Search module of OpenScholar. The module enables fast (<2 seconds) discovery of research papers using keyword-based full-text search and structured filtering (category, tags, university, department, year).

The design uses PostgreSQL Full-Text Search with GIN indexing for MVP and allows future migration to ElasticSearch.

2. Scope

The Search module is responsible for:

- Keyword-based search (title, abstract, keywords)
- Filtering by:
 - Category (fixed taxonomy)
 - Tags (free-form)
 - University
 - Department
 - Publication year
- Pagination of results
- Sorting (relevance, newest)

Out of Scope:

- Recommendation system
 - AI-based ranking (future enhancement)
-

3. Data Dependencies

Tables used:

- papers
- paper_versions
- paper_tags
- tags
- users
- universities
- departments

Key Conditions:

- Only approved papers should appear in search
 - Only latest published version should be used
 - Exclude soft-deleted papers
-

4. API Contract

4.1 Search Papers

Endpoint: GET /api/search

Query Parameters:

- q (string) – search keyword
- categoryId (number, optional)
- tagIds (array, optional)
- universityId (number, optional)
- departmentId (number, optional)
- year (number, optional)
- page (number, default: 1)
- limit (number, default: 10)
- sort (string: relevance | newest)

Example: GET /api/search?q=AI&categoryId=2&page=1&limit=10

Response: { "total": 120, "page": 1, "limit": 10, "results": [{ "paperId": "uuid", "title": "Paper Title", "abstract": "Short abstract", "keywords": ["AI", "ML"], "authors": [{ "name": "Author 1" }], "university": "XYZ University", "year": 2025, "pdfUrl": "url", "viewCount": 100 }] }

5. Search Strategy

5.1 Full-Text Search

Use PostgreSQL tsvector:

Fields indexed:

- title
- abstract

Query:

- plainto_tsquery OR websearch_to_tsquery

5.2 Keyword Search (Array)

Use GIN index on keywords array:

```
CREATE INDEX idx_keywords ON paper_versions USING GIN (keywords);
```

5.3 Combined Query Logic

Search should match:

- Full-text match (title, abstract)
 - OR keyword match
-

6. SQL Query (Reference)

```
SELECT p.id, pv.title, pv.abstract, pv.keywords, pv.pdf_url FROM papers p JOIN paper_versions pv ON  
pv.paper_id = p.id WHERE p.status = 'approved' AND p.is_deleted = FALSE AND pv.is_published = TRUE AND  
( to_tsvector('english', pv.title || ' ' || pv.abstract) @@ plainto_tsquery('english', $1) OR pv.keywords @>  
ARRAY[$1] ) ORDER BY pv.created_at DESC LIMIT $limit OFFSET $offset;
```

7. Filtering Logic

Apply filters conditionally:

- categoryId → p.category_id
 - tagIds → JOIN paper_tags
 - universityId → via user profile (paper creator)
 - departmentId → via user profile
 - year → EXTRACT(YEAR FROM pv.created_at)
-

8. Pagination

Use OFFSET-based pagination:

OFFSET = (page - 1) * limit

Future Improvement:

- Cursor-based pagination for large datasets

9. Sorting

- relevance → default (ts_rank)
 - newest → pv.created_at DESC
-

10. Service Logic

searchPapers()

Steps: 1. Parse query params 2. Build dynamic SQL filters 3. Execute search query 4. Fetch authors for each result 5. Fetch aggregated metrics (views) 6. Return structured response

11. Indexing Strategy

Required indexes:

```
CREATE INDEX idx_search_text ON paper_versions USING GIN (to_tsvector('english', title || ' ' || abstract));
```

```
CREATE INDEX idx_keywords ON paper_versions USING GIN (keywords);
```

```
CREATE INDEX idx_paper_status ON papers(status);
```

12. Performance Considerations

- Limit result size (default 10-20)
 - Avoid N+1 queries (batch fetch authors)
 - Use indexes for all filters
 - Cache frequent queries (future Redis)
-

13. Security Considerations

- Only approved and published papers should be visible
 - Prevent SQL injection (use Prisma parameterized queries)
-

14. Edge Cases

- Empty query (return trending or latest papers)
- No results found

- Invalid filter values
 - Large page numbers
-

15. Prisma Implementation Notes

- Use `prisma.$queryRaw` for full-text search
 - Use Prisma filters for structured conditions
 - Combine raw + ORM carefully
-

16. Folder Structure

`src/ app/api/search/route.ts`

`modules/search/service.ts` `modules/search/repository.ts`

17. Acceptance Criteria

- Search returns results within 2 seconds
 - Filters work correctly
 - Pagination returns correct data
 - Only approved papers are shown
 - Results include metadata, authors, and metrics
-

18. Future Enhancements

- ElasticSearch integration
 - Autocomplete suggestions
 - Search analytics
 - Ranking improvements
-

End of Document